

Reviewer Pack — Spectral Radius of SOS Moment Matrix Bounds Max-CUT Approxim...

Entry f7f5b0531d9e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 00:46:21 UTC

Spectral Radius of SOS Moment Matrix Bounds Max-CUT Approximation Ratio

Entry ID: f7f5b0531d9e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 00:46:21 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Moment Matrix

Statement:

For any max-CUT instance on n vertices, the spectral radius $\rho(M)$ of its degree- d SOS moment matrix satisfies $\rho(M) = \Theta(\sqrt{n})$ if and only if the SOS relaxation achieves approximation ratio $\Theta(1/\sqrt{n})$.

Rationale (proposer's reasoning):

The spectral properties of SOS moment matrices encode the curvature of the underlying polynomial optimization problem. Bounding the spectral radius provides a handle on the trade-off between relaxation degree and approximation quality, leveraging real algebraic geometry's tools for polynomial positivity.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0489c33372c9d0b3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_max_cut_instance(n):
    edges = []
    for i in range(n):
        for j in range(i + 1, n):
            if random.random() < 0.5:
                edges.append((i, j))
    return edges

def degree_matrix(edges, n):
    D = [[0] * n for _ in range(n)]
    for u, v in edges:
        D[u][u] += 1
        D[v][v] += 1
    return D

def adjacency_matrix(edges, n):
    A = [[0] * n for _ in range(n)]
    for u, v in edges:
        A[u][v] = 1
        A[v][u] = 1
    return A

def matrix_multiply(A, B):
    m = len(A)
    p = len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(len(B)):
                C[i][j] += A[i][k] * B[k][j]
    return C

def transpose(A):
    return [list(row) for row in zip(*A)]
```

```

def identity_matrix(n):
    I = [[0] * n for _ in range(n)]
    for i in range(n):
        I[i][i] = 1
    return I

def matrix_addition(A, B):
    m = len(A)
    n = len(A[0])
    C = [[A[i][j] + B[i][j] for j in range(n)] for i in range(m)]
    return C

def matrix_subtraction(A, B):
    m = len(A)
    n = len(A[0])
    C = [[A[i][j] - B[i][j] for j in range(n)] for i in range(m)]
    return C

def scalar_multiplication(A, c):
    return [[c * A[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_trace(A):
    return sum(A[i][i] for i in range(len(A)))

def matrix_power(A, k):
    if k == 1:
        return A
    elif k % 2 == 0:
        B = matrix_power(A, k // 2)
        return matrix_multiply(B, B)
    else:
        B = matrix_power(A, (k - 1) // 2)
        return matrix_multiply(matrix_multiply(B, B), A)

def spectral_radius(M):
    n = len(M)
    x = [Fraction(1, math.sqrt(n))] * n
    for _ in range(100): # Power iteration method
        y = matrix_multiply(M, x)
        x = scalar_multiplication(y, Fraction(1, matrix_norm(y)))
    return max(abs(x[i]) for i in range(n))

def matrix_norm(A):
    return math.sqrt(sum(sum(a[i][j] ** 2 for j in range(len(a[0]))) for a in A))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    edges = generate_max_cut_instance(n)
    D = degree_matrix(edges, n)
    A = adjacency_matrix(edges, n)
    M = matrix_addition(D, scalar_multiplication(A, -Fraction(1, 2)))

    rho_M = spectral_radius(M)
    c = rho_M / math.sqrt(n)
    approximation_ratio = c / math.sqrt(n)

```

```

return {
    "metric_name": "approximation_ratio",
    "metric_value": float(approximation_ratio),
    "instances_tested": 1,
    "conjecture_holds": abs(c - 1) < 0.1 and abs(approximation_ratio - 1) < 0.1,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 307)) # Default to first 30 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{first_failing_seed}\"")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bc789b2.py", line 124, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bc789b2.py", line 106, in run_trial
    rho_M = spectral_radius(M)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0bc789b2.py", line 89, in spectral_radius
    x = [Fraction(1, math.sqrt(n))] * n
          ~~~~~
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be ")
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to `TypeError` in `Fraction` initialization, preventing data collection | next:
Fix `Fraction` argument type conversion in test code to handle float inputs

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	89437
2	preregistration	ollama_remote	qwen3:8b	0	0	24169
3	novelty	ollama_remote	qwen3:8b	0	0	20977
4	novelty	ollama_remote	qwen3:8b	0	0	14438
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15114
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13129
7	judge	ollama_remote	qwen3:8b	0	0	15822

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 193086 ms total latency. Provider mix: `{'ollama_remote': 7}`

(full prompt+response transcripts available in `research/audit/f7f5b0531d9e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f7f5b0531d9e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f7f5b0531d9e.tar.gz` (if generated)